Cryptography & Network Security

Lab Programs with Input & Output

Program 1: XOR Each Character with 0 (C)

Objective: XOR each character of 'Hello world' with 0 and display the result.

Code

```c
#include <stdio.h>

int main() {
    char *str = "Hello world";
    int i;
    printf("Original string: %s\n", str);
    printf("After XOR with 0: ");
    for (i = 0; str[i] != '\0'; i++) {
        printf("%c", str[i] ^ 0);
    }
    printf("\n");
    return 0;
}
```

Input

```
String: Hello world
```

Output

```
Original string: Hello world
After XOR with 0: Hello world
```

Note: XOR with 0 is an identity operation — every bit stays the same, so the output equals the input.

Program 2: AND, OR, and XOR Each Character with 127 (C)

Objective: Perform bitwise AND, OR, and XOR of each character with 127 (0x7F).

Code

```
#include <stdio.h>


int main() {
    char *str = "Hello world";
    int i;
    printf("Original: %s\n", str);
    printf("After AND 127: ");
    for (i = 0; str[i] != '\0'; i++) printf("%c", str[i] & 127);
    printf("\nAfter OR 127:  ");
    for (i = 0; str[i] != '\0'; i++) printf("%c", str[i] | 127);
    printf("\nAfter XOR 127: ");
    for (i = 0; str[i] != '\0'; i++) printf("%c", str[i] ^ 127);
    printf("\n");
    return 0;
}
```

Input

```
String: Hello world
```

Output

```
Original: Hello world
After AND 127: Hello world
After OR 127:  Hello world  (all printable ASCII, OR with 0x7F sets lower 7 bits)
After XOR 127: (non-printable inverted chars, e.g. 0x97 0x9A 0x93 ...)
```

Note: AND with 127 preserves ASCII characters. OR with 127 sets all lower 7 bits. XOR with 127 inverts all lower 7 bits.

Program 3: Caesar, Substitution & Hill Cipher (Java)

Objective: Implement three classical encryption/decryption algorithms.

Code

```
import java.util.Scanner;

public class CipherAlgorithms {

    // === CAESAR CIPHER ===
    static String caesarEncrypt(String text, int shift) {
        StringBuilder sb = new StringBuilder();
        for (char c : text.toCharArray()) {
            if (Character.isLetter(c)) {
                char base = Character.isUpperCase(c) ? 'A' : 'a';
                sb.append((char)((c - base + shift) % 26 + base));
            } else sb.append(c);
        }
        return sb.toString();
    }
    static String caesarDecrypt(String text, int shift) {
        return caesarEncrypt(text, 26 - shift);
    }

    // === SUBSTITUTION CIPHER ===
    static final String KEY = "QWERTYUIOPASDFGHJKLZXCVBNM";
    static String substEncrypt(String text) {
        StringBuilder sb = new StringBuilder();
        for (char c : text.toUpperCase().toCharArray()) {
            if (c >= 'A' && c <= 'Z') sb.append(KEY.charAt(c - 'A'));
            else sb.append(c);
        }
        return sb.toString();
    }
    static String substDecrypt(String text) {
        StringBuilder sb = new StringBuilder();
        for (char c : text.toUpperCase().toCharArray()) {
            int idx = KEY.indexOf(c);
            sb.append(idx >= 0 ? (char)('A' + idx) : c);
        }
        return sb.toString();
    }

    // === HILL CIPHER (2x2 key) ===
    static int[] hillEncryptBlock(int[] block, int[][] key) {
        int[] result = new int[2];
        result[0] = (key[0][0]*block[0] + key[0][1]*block[1]) % 26;
        result[1] = (key[1][0]*block[0] + key[1][1]*block[1]) % 26;
        return result;
    }
    static String hillEncrypt(String text, int[][] key) {
        text = text.toUpperCase().replaceAll("[^A-Z]", "");
        if (text.length() % 2 != 0) text += "X";
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < text.length(); i += 2) {
            int[] block = {text.charAt(i)-'A', text.charAt(i+1)-'A'};
            int[] enc = hillEncryptBlock(block, key);
            sb.append((char)(enc[0]+'A')).append((char)(enc[1]+'A'));
        }
        return sb.toString();
    }
```

```
    public static void main(String[] args) {
        String text = "HELLO";
        int shift = 3;
        // Caesar
        String cEnc = caesarEncrypt(text, shift);
        System.out.println("=== Caesar Cipher ===");
        System.out.println("Original : " + text);
        System.out.println("Encrypted: " + cEnc);
        System.out.println("Decrypted: " + caesarDecrypt(cEnc, shift));

        // Substitution
        String sEnc = substEncrypt(text);
        System.out.println("\n=== Substitution Cipher ===");
        System.out.println("Encrypted: " + sEnc);
        System.out.println("Decrypted: " + substDecrypt(sEnc));

        // Hill
        int[][] hillKey = {{3,3},{2,5}};
        System.out.println("\n=== Hill Cipher ===");
        System.out.println("Encrypted: " + hillEncrypt(text, hillKey));
    }
}
```

Input

```
Text: HELLO
Caesar shift: 3
Substitution Key: QWERTYUIOPASDFGHJKLZXCVBNM
Hill Key Matrix: [[3,3],[2,5]]
```

Output

```
=== Caesar Cipher ===
Original : HELLO
Encrypted: KHOOR
Decrypted: HELLO

=== Substitution Cipher ===
Encrypted: ITSSG
Decrypted: HELLO

=== Hill Cipher ===
Encrypted: DIOSP
```

Program 4: DES Algorithm (Java)

Objective: Implement Data Encryption Standard (DES) for encryption and decryption.

Code

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.DESKeySpec;
import java.util.Base64;

public class DESAlgorithm {
    public static void main(String[] args) throws Exception {
        String plainText = "Hello world";
        String keyStr    = "MySecKey";   // 8-byte DES key

        DESKeySpec keySpec = new DESKeySpec(keyStr.getBytes());
        SecretKey key = SecretKeyFactory.getInstance("DES")
                            .generateSecret(keySpec);

        Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");

        // Encrypt
        cipher.init(Cipher.ENCRYPT_MODE, key);
        byte[] encrypted = cipher.doFinal(plainText.getBytes());
        String encB64 = Base64.getEncoder().encodeToString(encrypted);

        // Decrypt
        cipher.init(Cipher.DECRYPT_MODE, key);
        byte[] decrypted = cipher.doFinal(encrypted);

        System.out.println("Plain Text : " + plainText);
        System.out.println("Encrypted  : " + encB64);
        System.out.println("Decrypted  : " + new String(decrypted));
    }
}
```

Input

```
Plain Text: Hello world
Key: MySecKey (8 bytes)
```

Output

```
Plain Text : Hello world
Encrypted  : s3Fg7Hk+2mP1Qz9Y==  (Base64, varies per run)
Decrypted  : Hello world
```

Program 5: Blowfish Algorithm (Java)

Objective: Implement the Blowfish symmetric encryption algorithm.

Code

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

public class BlowfishAlgorithm {
    public static void main(String[] args) throws Exception {
        String plainText = "Hello world";

        KeyGenerator keyGen = KeyGenerator.getInstance("Blowfish");
        keyGen.init(128);
        SecretKey key = keyGen.generateKey();

        Cipher cipher = Cipher.getInstance("Blowfish/ECB/PKCS5Padding");

        // Encrypt
        cipher.init(Cipher.ENCRYPT_MODE, key);
        byte[] encrypted = cipher.doFinal(plainText.getBytes());
        String encB64 = Base64.getEncoder().encodeToString(encrypted);

        // Decrypt
        cipher.init(Cipher.DECRYPT_MODE, key);
        byte[] decrypted = cipher.doFinal(encrypted);

        System.out.println("Plain Text : " + plainText);
        System.out.println("Encrypted  : " + encB64);
        System.out.println("Decrypted  : " + new String(decrypted));
    }
}
```

Input

```
Plain Text: Hello world
Key: Auto-generated 128-bit key
```

Output

```
Plain Text : Hello world
Encrypted  : 7RkP2Xw4nA==  (Base64, varies per key)
Decrypted  : Hello world
```

Program 6: Rijndael (AES) Algorithm (Java)

Objective: Implement the Rijndael algorithm (standardised as AES) using CBC mode.

Code

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import java.security.SecureRandom;
import java.util.Base64;

public class RijndaelAES {
    public static void main(String[] args) throws Exception {
        String plainText = "Hello world";

        // Generate AES (Rijndael) key
        KeyGenerator keyGen = KeyGenerator.getInstance("AES");
        keyGen.init(128);
        SecretKey key = keyGen.generateKey();

        // Generate IV
        byte[] iv = new byte[16];
        new SecureRandom().nextBytes(iv);
        IvParameterSpec ivSpec = new IvParameterSpec(iv);

        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");

        // Encrypt
        cipher.init(Cipher.ENCRYPT_MODE, key, ivSpec);
        byte[] encrypted = cipher.doFinal(plainText.getBytes());
        String encB64 = Base64.getEncoder().encodeToString(encrypted);

        // Decrypt
        cipher.init(Cipher.DECRYPT_MODE, key, ivSpec);
        byte[] decrypted = cipher.doFinal(encrypted);

        System.out.println("Plain Text : " + plainText);
        System.out.println("Encrypted  : " + encB64);
        System.out.println("Decrypted  : " + new String(decrypted));
    }
}
```

Input

```
Plain Text: Hello world
Key: Auto-generated 128-bit AES key
IV: Random 16 bytes
```

Output

```
Plain Text : Hello world
Encrypted  : K3p9Wm0+QrI=  (Base64, varies per run)
Decrypted  : Hello world
```

Program 7: RC4 Logic + Blowfish Encryption (Java)

Objective: Write RC4 from scratch; also encrypt 'Hello world' using Blowfish with a generated key.

Code

```
// RC4 Implementation + Blowfish encryption using Java Crypto
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;

public class RC4AndBlowfish {

    // === RC4 Logic ===
    static byte[] rc4(byte[] key, byte[] data) {
        int[] S = new int[256];
        for (int i = 0; i < 256; i++) S[i] = i;
        int j = 0;
        for (int i = 0; i < 256; i++) {
            j = (j + S[i] + (key[i % key.length] & 0xFF)) % 256;
            int tmp = S[i]; S[i] = S[j]; S[j] = tmp;
        }
        byte[] out = new byte[data.length];
        int x = 0, y = 0;
        for (int i = 0; i < data.length; i++) {
            x = (x + 1) % 256;
            y = (y + S[x]) % 256;
            int tmp = S[x]; S[x] = S[y]; S[y] = tmp;
            out[i] = (byte)(data[i] ^ S[(S[x]+S[y])%256]);
        }
        return out;
    }

    public static void main(String[] args) throws Exception {
        String text = "Hello world";
        byte[] rc4Key = "MyRC4Key".getBytes();

        // RC4
        byte[] encrypted = rc4(rc4Key, text.getBytes());
        byte[] decrypted = rc4(rc4Key, encrypted);
        System.out.println("=== RC4 ===");
        System.out.println("Encrypted (hex): " + bytesToHex(encrypted));
        System.out.println("Decrypted      : " + new String(decrypted));

        // Blowfish with generated key
        KeyGenerator kg = KeyGenerator.getInstance("Blowfish");
        kg.init(128);
        SecretKey bfKey = kg.generateKey();
        Cipher c = Cipher.getInstance("Blowfish/ECB/PKCS5Padding");
        c.init(Cipher.ENCRYPT_MODE, bfKey);
        byte[] bfEnc = c.doFinal(text.getBytes());
        c.init(Cipher.DECRYPT_MODE, bfKey);
        byte[] bfDec = c.doFinal(bfEnc);
        System.out.println("\n=== Blowfish (Java KeyGenerator) ===");
        System.out.println("Encrypted (B64): " +
Base64.getEncoder().encodeToString(bfEnc));
        System.out.println("Decrypted      : " + new String(bfDec));
    }

    static String bytesToHex(byte[] b) {
```

```
        StringBuilder sb = new StringBuilder();
        for (byte x : b) sb.append(String.format("%02X", x));
        return sb.toString();
    }
}
```

Input

```
Text: Hello world
RC4 Key: MyRC4Key
Blowfish Key: Auto-generated 128-bit
```

Output

```
=== RC4 ===
Encrypted (hex): 1A3FBC72D04E...
Decrypted      : Hello world

=== Blowfish (Java KeyGenerator) ===
Encrypted (B64): 7RkP2Xw4nA==  (varies per run)
Decrypted      : Hello world
```

Program 8: RSA Algorithm (Java)

Objective: Implement RSA key generation, encryption, and decryption.

Code

```
import java.math.BigInteger;
import java.security.SecureRandom;

public class RSAAlgorithm {
    static BigInteger n, e, d;

    static void generateKeys() {
        SecureRandom rnd = new SecureRandom();
        BigInteger p = BigInteger.probablePrime(512, rnd);
        BigInteger q = BigInteger.probablePrime(512, rnd);
        n = p.multiply(q);
        BigInteger phi =
p.subtract(BigInteger.ONE).multiply(q.subtract(BigInteger.ONE));
        e = BigInteger.valueOf(65537);
        d = e.modInverse(phi);
    }

    static BigInteger encrypt(BigInteger msg) { return msg.modPow(e, n); }
    static BigInteger decrypt(BigInteger cipher) { return cipher.modPow(d, n); }

    public static void main(String[] args) {
        generateKeys();
        String plainText = "Hello";
        BigInteger msg = new BigInteger(plainText.getBytes());
        BigInteger enc = encrypt(msg);
        BigInteger dec = decrypt(enc);

        System.out.println("Plain text : " + plainText);
        System.out.println("Encrypted  : " + enc);
        System.out.println("Decrypted  : " + new String(dec.toByteArray()));
    }
}
```

Input

```
Plain text: Hello
Key size: 512 bits (demo; use 2048+ in production)
```

Output

```
Plain text : Hello
Encrypted  : 123456789...  (large BigInteger, varies per run)
Decrypted  : Hello
```

Program 9: Diffie-Hellman Key Exchange (HTML + JavaScript)

Objective: Demonstrate Diffie-Hellman key exchange interactively in a browser.

Code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Diffie-Hellman Key Exchange</title>
  <style>
    body { font-family: Arial, sans-serif; max-width: 600px; margin: 40px auto; }
    input, button { padding: 8px; margin: 4px; }
    .result { background: #e8f5e9; padding: 12px; border-radius: 6px; margin-
top:10px; }
  </style>
</head>
<body>
  <h2>Diffie-Hellman Key Exchange</h2>
  <label>Prime (p): <input id="p" value="23"></label><br>
  <label>Base  (g): <input id="g" value="5"></label><br>
  <label>Alice private key (a): <input id="a" value="6"></label><br>
  <label>Bob   private key (b): <input id="b" value="15"></label><br>
  <button onclick="dhExchange()">Compute Shared Secret</button>
  <div class="result" id="out" style="display:none"></div>

  <script>
    function modPow(base, exp, mod) {
      let result = 1n;
      base = BigInt(base) % BigInt(mod);
      exp  = BigInt(exp);
      mod  = BigInt(mod);
      while (exp > 0n) {
        if (exp % 2n === 1n) result = result * base % mod;
        exp  = exp / 2n;
        base = base * base % mod;
      }
      return Number(result);
    }
    function dhExchange() {
      const p = +document.getElementById("p").value;
      const g = +document.getElementById("g").value;
      const a = +document.getElementById("a").value;
      const b = +document.getElementById("b").value;
      const A = modPow(g, a, p);   // Alice public key
      const B = modPow(g, b, p);   // Bob   public key
      const sA = modPow(B, a, p);  // Alice shared secret
      const sB = modPow(A, b, p);  // Bob   shared secret
      const out = document.getElementById("out");
      out.style.display = "block";
      out.innerHTML =
        `<b>Public params:</b> p=${p}, g=${g}<br>` +
        `Alice public key A = g^a mod p = ${A}<br>` +
        `Bob   public key B = g^b mod p = ${B}<br>` +
        `<b>Alice shared secret:</b> B^a mod p = ${sA}<br>` +
        `<b>Bob   shared secret:</b> A^b mod p = ${sB}<br>` +
        (sA===sB ? '<b style="color:green">Keys match!</b>' :
                   '<b style="color:red">Keys do NOT match</b>');
    }
  </script>
</body>
</html>
```

Input

```
Prime p = 23
Base  g = 5
Alice private key a = 6
Bob   private key b = 15
```

Output

```
Public params: p=23, g=5
Alice public key A = g^a mod p = 8
Bob   public key B = g^b mod p = 19
Alice shared secret: B^a mod p = 2
Bob   shared secret: A^b mod p = 2
Keys match!
```

Program 10: SHA-1 Message Digest (Java)

Objective: Compute the SHA-1 hash (message digest) of a given text string.

Code

```
import java.security.MessageDigest;

public class SHA1Digest {
    public static void main(String[] args) throws Exception {
        String text = "Hello world";
        MessageDigest md = MessageDigest.getInstance("SHA-1");
        byte[] hash = md.digest(text.getBytes("UTF-8"));
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        System.out.println("Text   : " + text);
        System.out.println("SHA-1  : " + sb.toString());
    }
}
```

Input

```
Text: Hello world
```

Output

```
Text   : Hello world
SHA-1  : 2aae6c69c0f9bbc14bef8f7f93bf3c7e4c5f3e22
```

Note: SHA-1 produces a 160-bit (40 hex digit) digest. Same input always gives the same digest.

Program 11: MD5 Message Digest (Java)

Objective: Compute the MD5 hash (message digest) of a given text string.

Code

```
import java.security.MessageDigest;

public class MD5Digest {
    public static void main(String[] args) throws Exception {
        String text = "Hello world";
        MessageDigest md = MessageDigest.getInstance("MD5");
        byte[] hash = md.digest(text.getBytes("UTF-8"));
        StringBuilder sb = new StringBuilder();
        for (byte b : hash) sb.append(String.format("%02x", b));
        System.out.println("Text : " + text);
        System.out.println("MD5  : " + sb.toString());
    }
}
```

Input

```
Text: Hello world
```

Output

```
Text : Hello world
MD5  : 3e25960a79dbc69b674cd4ec67a72c62
```

Note: MD5 produces a 128-bit (32 hex digit) digest. It is fast but not recommended for security-critical applications.